B19 Testing in depth

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2

B19 Testing in depth 3

B19-L1 How pytest finds, runs and reports your tests 3

B19-L2 Fixtures, shared setup without shared state 8

B19-L3 Parametrize, one test and many readable cases 13

B19-L4 Testing pandas and NumPy code 18

B19-L5 Testing statistical and random code 23

B19-L6 Testing machine learning pipelines 27

B19-L7 Isolate I/O with fakes, monkeypatch and mocks 32

B19-L8 Coverage, markers and a test suite you can trust in CI 37

Track B best-practice checklist 44

Appendix: solutions and answer keys 45

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B19 Testing in depth

Roadmap: Prerequisite for all months. pytest from the inside: fixtures, parametrize, testing pandas, statistics and ML, mocks, coverage and CI.

B19-L1 How pytest finds, runs and reports your tests

Concept

pytest only runs what it **collects**. With default settings in pytest 9, it looks for:

- files named `test_*.py` or `*_test.py`,
- functions in those files whose names start with `test`,
- methods starting with `test` inside classes whose names start with `Test`, as long as the class has no `__init__`.

A function called `check_weights` in `validation.py` never runs. pytest does not warn you about it.

Use plain `assert`. pytest rewrites assert statements, so a failure shows the values on both sides and the call that produced them:

```
E       assert 0.3333333333333333 == 0.33
E        +  where 0.3333333333333333 = share(1, 3)
```

Test errors on purpose with `pytest.raises`. Pass `match=` a regular expression. pytest searches the error message for it, so a `ValueError` raised for a different reason fails the test:

```
with pytest.raises(ValueError, match="negative weight"):
    normalise([2.0, -1.0])
```

If nothing is raised, the test fails with "DID NOT RAISE". An exception of another type passes straight through and fails the test too.

Run less while you work:

- `pytest tests/test_share.py::test_zero_total` runs one test by its **node ID**.
- `pytest -k "zero and not slow"` runs tests whose names match the expression.
- `pytest -x` stops at the first failure. `pytest --lf` reruns only the tests that failed last time.

The exit code tells CI what happened: 0 when everything passed, 1 when a test failed, and 5 when pytest collected no tests at all. Set where tests live once, in `pyproject.toml`: `testpaths = ["tests"]` under `[tool.pytest.ini_options]`, or under `[tool.pytest]`, which pytest 9 also reads.

Why it matters

A team kept 30 data checks in `validation_checks.py`, as functions named `validate_*`. The rest of the suite lived in properly named files, so every CI run was green. pytest never collected the 30 checks. A broken weight cap shipped in four monthly reports before anyone noticed. Renaming the file and the functions turned up two failures on the first run.

Worked example: weak code

```
1  # validation_checks.py
2  def check_negative_weights():
3      try:
4          normalise([2.0, -1.0])
3          print("no error!")
4      except Exception:
7          print("ok")
```

1. **Line 1.** The file name does not match test_*.py, so pytest never collects it.
2. **Line 2.** The function name does not start with test either.
3. **Line 5.** A print never fails a test. Nobody reads it in CI.
4. **Line 6.** Any error passes, including a TypeError from a typo.

Worked example: strong code

```
1  # tests/test_normalise.py
2  import pytest
3
4  from survey.weights import normalise
5
6
7  def test_weights_sum_to_one() -> None:
8      assert sum(normalise([1.0, 3.0])) == pytest.approx(1.0)
9
10
2  def test_negative_weight_raises() -> None:
3      with pytest.raises(ValueError, match="negative weight"):
13         normalise([2.0, -1.0])
```

1. **Line 1.** A collected file name, in the folder that testpaths names.
2. **Line 11.** A collected function, named for the behaviour it checks.

3. **Line 12.** Fails if nothing is raised, if another type is raised, or if the message differs.

Name files and functions so pytest collects them, assert instead of printing, and pin expected errors with pytest.raises and match.

Recap card

- **What pytest collects by default** Files test_*.py or *_test.py, functions starting with test, and test methods in Test classes without **init**.
- **Test that code raises the right error** with pytest.raises(ValueError, match="pattern"): call(). match searches the message with a regular expression.
- **Run one test, or only failures** pytest path::test_name for one test. -k expr selects by name. -x stops at the first failure. --lf reruns last failures.
- **pytest exit codes** 0 all passed, 1 a test failed, 5 no tests collected.

Practice

Exercise 1. Multiple choice (level 1)

With default settings, which of these does pytest 9 collect and run?

- a) `def check_weights():` in `tests/test_weights.py`
- b) `def test_weights():` in `tests/weight_checks.py`
- c) `def test_weights():` in `tests/weights_test.py`
- d) `def test_total(self):` in `class WeightTests:` in `tests/test_weights.py`

Why?

- r1) Both the file name (`*_test.py`) and the function name (`test*`) match the default patterns.
- r2) pytest runs every function in the tests folder.

Exercise 2. Predict the output (level 2)

What does this print? (pytest 9)

```
 1 import pytest
 2
 3
 4 def check(total):
 5     if total < 0:
 6         raise ValueError(f"total must be >= 0, got {total}")
 7
 8
 9 with pytest.raises(ValueError, match=r"got -\d"):
10     check(-5)
```

```
print("first ok")

try:
    with pytest.raises(ValueError, match="must be > 0"):
        check(-5)
except AssertionError:
    print("pattern did not match")
```

Exercise 3. Fill the gap (level 2)

Run only the tests whose names contain `zero`, and stop at the first failure.

```
pytest -k _____(1) -_____(2)
```

Exercise 4. Write the function (level 3)

`normalise(weights)` divides each weight by the total. It must raise `ValueError` with a message containing `negative weight` for any negative weight, and `ValueError` containing `total weight is 0` when the weights sum to 0.

Write two tests with `pytest.raises` and `match=`:

- `test_negative_weight_raises()` calls `normalise([2.0, -1.0])`.
- `test_zero_total_raises()` calls `normalise([0.0, 0.0])`.

The checker runs your tests against a correct `normalise`, which must pass, and against three broken versions, which your tests must catch.

Your code must pass these checks:

- Your tests must pass for a correct normalise.
- test_negative_weight_raises must fail when normalise accepts a negative weight. A try/except that passes on no error cannot catch that.
- test_zero_total_raises must expect ValueError, not any exception. A ZeroDivisionError is a bug.
- Add `match=` with the expected message, so a ValueError raised for another reason fails the test.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Scale weights so they sum to 1.
def normalise(weights: list[float]) -> list[float]: ...

def test_negative_weight_raises() -> None: ...

def test_zero_total_raises() -> None: ...
```

Starter code:

```
import pytest

```

```
 3
 4 def normalise(weights: list[float]) -> list[float]:
 5     """Scale weights so they sum to 1."""
 6     for i, w in enumerate(weights):
 7         if w < 0:
 8             raise ValueError(f"negative weight at index {i}")
 9     total = sum(weights)
10     if total == 0:
11         raise ValueError("total weight is 0")
12     return [w / total for w in weights]
13
14
15 def test_negative_weight_raises():
16     try:
17         normalise([2.0, -1.0])
18     except ValueError:
19         pass
20
21
22 def test_zero_total_raises():
23     try:
24         normalise([0.0, 0.0])
25     except Exception:
26         pass
```

Exercise 5. Put in order (level 3)

A test fails in CI. Put the steps to fix it in order.

- Run the whole suite, then push.
- Copy the failing test's node ID from the CI log.
- Run that one test locally: `pytest tests/test_share.py::test_zero_total`.
- Change the code until that test passes.
- Run `pytest --lf` to rerun every test that failed last time.
- Read the assertion output: the values on both sides and the call that produced them.

Sources

- Default collection: test_*.py and *_test.py files, test-prefixed functions, Test-prefixed classes without **init** (with a PytestCollectionWarning). (verified: Run on pytest 9.2.0 in this repository's scratch environment.)
- pytest.raises fails with 'DID NOT RAISE' when nothing is raised, lets other exception types through, and raises AssertionError when match does not find the pattern. (verified: Run on pytest 9.0.2 (Pyodide pin) and 9.2.0.)

- Exit codes 0, 1 and 5; -k, -x, --lf and node IDs; testpaths read from [tool.pytest.ini_options] and from [tool.pytest]. (verified: Run on pytest 9.0.2 and 9.2.0.)

B19-L2 Fixtures, shared setup without shared state

Concept

A **fixture** is a function decorated with `@pytest.fixture`. A test asks for it by naming it as a parameter, and pytest calls the fixture and passes in the result:

```
@pytest.fixture
def respondents() -> pd.DataFrame:
    return pd.DataFrame({"respondent_id": ["R1", "R2"], "final_weight":
    [1.0, 3.0]})


def test_total_weight(respondents: pd.DataFrame) -> None:
    assert respondents["final_weight"].sum() == 4.0
```

By default a fixture runs once **per test**, so every test gets a fresh object. A test can change it without touching any other test.

Use `yield` when something must be cleaned up. Code before `yield` is setup. Code after it is teardown, and pytest runs it even when the test fails.

`scope="module"` or `scope="session"` builds the object once and shares it. Use that only for things that are expensive and that no test changes, such as the SparkSession in lesson A9-L1. A shared object that one test changes makes results depend on the order tests run in.

Put fixtures that several files use in `conftest.py`. Tests in that folder and its subfolders can use them without an import.

pytest ships built-in fixtures. `tmp_path` gives each test a fresh temporary directory as a `pathlib.Path`. `monkeypatch` changes settings and undoes the change afterwards (lesson B19-L7).

When tests need many slightly different inputs, write a **factory fixture**. It returns a function that builds the data, with defaults for everything, so each test states only what matters to it:

```
@pytest.fixture
def make_respondents():
    def make(weights: list[float]) -> pd.DataFrame:
        ids = [f"R{i}" for i in range(1, len(weights) + 1)]
        return pd.DataFrame({"respondent_id": ids, "final_weight":
    weights})
```

```
    return make
```

Why it matters

A QA test suite shared one session-scoped DataFrame of 500 respondents across 40 tests. One test capped the weights in place to check the cap function. On a laptop the tests ran in file order and passed. In CI a plugin ran them in random order, and three weight-total tests failed about one run in four. The team spent two days calling the failures "flaky" before they found the in-place change. Function scope fixed it in one line.

Worked example: weak code

```
import pandas as pd

RESPONDENTS = pd.DataFrame({"respondent_id": ["R1", "R2"],
    "final_weight": [1.0, 3.0]})


def test_cap_weights():
    RESPONDENTS.loc[RESPONDENTS["final_weight"] > 2, "final_weight"] = 2.0
    assert RESPONDENTS["final_weight"].max() == 2.0


def test_total_weight():
    assert RESPONDENTS["final_weight"].sum() == 4.0
```

1. **Line 3.** One module-level frame, shared by every test in the file.
2. **Line 7.** Changes the shared frame in place. Every later test sees 2.0 instead of 3.0.
3. **Line 12.** Passes alone, fails after test_cap_weights: a test that depends on order.

Worked example: strong code

```
import pandas as pd
import pytest


@pytest.fixture
def respondents() -> pd.DataFrame:
    return pd.DataFrame({"respondent_id": ["R1", "R2"], "final_weight":
    [1.0, 3.0]})


def test_cap_weights(respondents: pd.DataFrame) -> None:
    capped = cap_weights(respondents, cap=2.0)
    assert capped["final_weight"].max() == 2.0


```

```
15 def test_total_weight(respondents: pd.DataFrame) -> None:
16     assert respondents["final_weight"].sum() == 4.0
```

1. **Line 5.** Function scope by default: a fresh frame for every test.
2. **Line 10.** The test asks for the fixture by naming it as a parameter.
3. **Line 11.** The function under test returns a new frame. Nothing shared changes.

Fixtures give each test fresh data by default. Share a fixture only when it is expensive and nothing changes it.

Recap card

- **What a fixture is** A function decorated with @pytest.fixture. A test gets its value by naming it as a parameter.
- **Fixture setup and teardown** Code before yield is setup. Code after yield is teardown, and it runs even when the test fails.
- **When to widen a fixture's scope** Only for expensive objects that no test changes. A shared object that a test changes makes results depend on test order.
- **Factory fixture** A fixture that returns a function building fresh data with defaults, so each test states only what matters.
- **Share fixtures across files** Put them in conftest.py. Tests in that folder and below use them without an import.

Practice

Exercise 1. Multiple choice (level 1)

A fixture returns a three-row DataFrame. Several tests change values in it. Which scope should it have?

- a) The default, function scope
- b) `scope="module"`
- c) `scope="session"`
- d) A module-level variable instead of a fixture

Why?

- r1) Each test gets its own fresh frame, so one test's changes cannot reach another.
- r2) Function scope makes tests run faster.

Exercise 2. Put in order (level 2)

`conftest.py` holds this fixture. `test_fix.py` has two tests that both use it and print their own names. You run `pytest -s tests/test_fix.py` (`-s` shows prints). Put the printed lines in order.

```
@pytest.fixture
def weights():
    print("setup")
    yield [1.0, 2.0]
    print("teardown")


def test_sum(weights):
    print("test_sum")
    assert sum(weights) == 3.0


def test_len(weights):
    print("test_len")
    assert len(weights) == 2
```

- teardown (first time)
- test_len
- teardown (second time)
- setup (second time)
- setup (first time)
- test_sum

Exercise 3. Fill the gap (level 2)

Open one DuckDB connection for all the tests in a file, and close it after the last one.

```
@pytest._____(1)(scope="_____(2)")
def con():
    connection = duckdb.connect()
    _____(3) connection
    connection.close()
```

Exercise 4. Write the function (level 3)

Replace the shared module-level frame with a factory fixture.

- `make_respondents` is a fixture that returns a function `make(weights, in_scope=None)`. It builds a new DataFrame with `respondent_id` (`"R1"`, `"R2"`, ...), `final_weight` (the weights, as floats) and `in_scope` (the given booleans, or all `True` when `in_scope` is `None`).

- `test_in_scope_weight_skips_out_of_scope(make_respondents)` builds two rows with weights 1.0 and 3.0, where only the first is in scope, and asserts that `in_scope_weight(df)` is 1.0.

Your code must pass these checks:

- make([1.0, 2.0]) must give columns respondent_id, final_weight and in_scope, with IDs R1, R2, float weights and in_scope all True.
- Pass the in_scope list through when it is given.
- Each call must build a new frame, so a change to one never reaches another.
- Your test must pass for a correct in_scope_weight.
- Your test must fail when in_scope_weight also counts the out-of-scope row. Mark the 3.0 row out of scope and assert 1.0.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Total final_weight of in-scope rows.
def in_scope_weight(df: pd.DataFrame) -> float: ...

# Factory for small respondent frames with sensible defaults.
def make_respondents(): ...

def test_in_scope_weight_skips_out_of_scope(
    make_respondents,
) -> None: ...
```

Starter code:

```python
import pandas as pd
import pytest


def in_scope_weight(df: pd.DataFrame) -> float:
    """Total final_weight of in-scope rows."""
    return float(df.loc[df["in_scope"], "final_weight"].sum())


RESPONDENTS = pd.DataFrame(
    {"respondent_id": ["R1", "R2"], "final_weight": [1.0, 3.0],
   "in_scope": [True, False]}
)


def test_in_scope_weight_skips_out_of_scope():
    assert in_scope_weight(RESPONDENTS) == 1.0
```

Exercise 5. Spot the bug (level 4)

`test_total_weight` passes on its own and fails when the whole file runs. Tap the **two** lines that together make the tests depend on each other.

```
import pandas as pd
import pytest


@pytest.fixture(scope="module")
def respondents():
    return pd.DataFrame({"id": ["R1", "R2"], "w": [1.0, 3.0]})


def test_cap_weights(respondents):
    respondents.loc[respondents["w"] > 2, "w"] = 2.0
    assert respondents["w"].max() == 2.0


def test_total_weight(respondents):
    assert respondents["w"].sum() == 4.0
```

Write down the line number of the bug.

Sources

- Function scope is the default; code after yield runs after a failing test; conftest.py fixtures reach their folder and subfolders, and a test elsewhere gets "fixture not found". (verified: Run on pytest 9.2.0 in this repository's scratch environment.)
- A module-scoped fixture changed in place by one test makes a later test fail, and the same tests pass in the reverse order. (verified: Run on pytest 9.2.0 with pandas 3.0.6.)
- Calling a fixture directly fails; fixture.**wrapped** returns the undecorated function in pytest 9.0.2 and 9.2.0. (verified: Run in the dev and Pyodide-pin environments. The E4 checker relies on **wrapped**.)

B19-L3 Parametrize, one test and many readable cases

Concept

`@pytest.mark.parametrize` runs one test function once per row of a table. Each row becomes its own test, with its own pass or fail:

```
@pytest.mark.parametrize(
    ("age", "expected"),
    [
        pytest.param(34, "16-34", id="below-35"),
```

```
        pytest.param(35, "35-54", id="at-35"),
        pytest.param(None, None, id="missing"),
    ],
)
def test_age_band(age: int | None, expected: str | None) -> None:
    assert age_band(age) == expected
```

pytest names each case from its values, such as `test_share[1-4-0.25]`. Give a case a name with `pytest.param(..., id="missing")`, so a failure reads `test_age_band[missing]`.

Choose rows on purpose. For every threshold, test the value just below it, the value on it, and a missing value. Most banding bugs are a `<` that should be a `<=`.

Mark a known bug on one row with `pytest.param(..., marks=pytest.mark.xfail(reason="bug #41", strict=True))`. The row is expected to fail. With `strict=True`, the run fails as soon as the row starts passing, which tells you to remove the mark once the bug is fixed.

Two `parametrize` decorators on one test run every combination: two values times three values make six tests.

To run every test that uses a fixture once per value, give the fixture `params`. Read the current value from the built-in `request` fixture:

```
@pytest.fixture(params=["pandas", "polars"])
def backend(request: pytest.FixtureRequest) -> str:
    return request.param
```

Keep the test body free of `if` statements that pick a behaviour per row. When some rows should raise and others return a value, write two tables and two tests.

Why it matters

An age banding function had one test, for age 40. A refactor changed `age < 35` to `age <= 35`, and every 35-year-old moved into the younger band. In a tracker with 800 respondents a wave, about 16 people moved band each wave. The 16-34 share rose by two points, and the client asked why. A parametrized test with rows 34 and 35 fails on that change before review.

Worked example: weak code

```
def test_age_band():
    for age, expected in [(20, "16-34"), (40, "35-54"), (70, "55+")]:
        if age is None:
            assert age_band(age) is None
        else:
```

```
6             assert age_band(age) == expected
```

1. **Line 2.** One test for three cases: the first failure hides the rest, and the report does not say which case broke.
2. **Line 3.** Branching inside the test. The table never contains None, so this branch never runs.

Worked example: strong code

```
1  import pytest
2
3
4  @pytest.mark.parametrize(
5      ("age", "expected"),
6      [
7          pytest.param(34, "16-34", id="below-35"),
(1)        pytest.param(35, "35-54", id="at-35"),
9          pytest.param(54, "35-54", id="below-55"),
10         pytest.param(55, "55+", id="at-55"),
(2)        pytest.param(None, None, id="missing"),
12     ],
13 )
14 def test_age_band(age: int | None, expected: str | None) -> None:
(3)    assert age_band(age) == expected
```

1. **Line 8.** Each threshold is tested on both sides. This row catches a < that became <=.
2. **Line 11.** The missing value gets its own named row.
3. **Line 15.** One assertion, no branching. Every row runs and reports separately.

One row per case, with both sides of every threshold and the missing value, each named so a failure explains itself.

Recap card

- **Run one test over a table of cases** @pytest.mark.parametrize(("age", "expected"), [...]). Each row is its own test.
- **Name a parametrized case** pytest.param(34, "16-34", id="below-35") reports as test_age_band[below-35].
- **Mark a known bug on one row** pytest.param(..., marks=pytest.mark.xfail(reason=..., strict=True)). strict fails the run once the row passes.
- **Run every test once per implementation** @pytest.fixture(params=[...]) and return request.param.
- **Rows to pick for a threshold** Just below it, on it, and a missing value.

Practice

Exercise 1. Predict the output (level 1)

How many tests does pytest collect from this file?

```python
import pytest


@pytest.mark.parametrize("total", [1, 4, 10])
@pytest.mark.parametrize("part", [0, 1])
def test_grid(part, total):
    assert part <= total


@pytest.fixture(params=["pandas", "polars"])
def backend(request):
    return request.param


def test_backend(backend):
    assert backend in {"pandas", "polars"}
```

Exercise 2. Multiple choice (level 2)

A row is marked `pytest.param(1, 3, 0.33, marks=pytest.mark.xfail(reason="rounding bug #41", strict=True))`. A colleague fixes the rounding bug. What happens on the next run?

- a) The row passes and the suite stays green.
- b) The row is reported as XPASS(strict) and the run fails.
- c) pytest skips the row.
- d) pytest removes the mark itself.

Why?

- r1) strict=True turns an unexpected pass into a failure, so the stale mark gets removed.
- r2) xfail rows always fail.

Exercise 3. Fill the gap (level 2)

Name the row for age 34, so a failure reads `test_age_band[just-below-35]`.

```python
@pytest.mark.parametrize(
    ("age", "expected"),
    [
        pytest._____(1)(34, "16-34", _____(2)="just-below-35"),
        pytest.param(35, "35-54", id="at-35"),
    ],
)
```

```python
def test_age_band(age, expected):
    assert age_band(age) == expected
```

Exercise 4. Write the function (level 3)

`age_band(age)` returns `"16-34"` under 35, `"35-54"` under 55, `"55+"` from 55, and `None` for a missing age.

Replace the loop with a parametrized test `test_age_band(age, expected)`. Your table must include the ages 34, 35, 54 and 55, and `None`, each with its correct expected band.

The checker reads your table, runs every row against a correct `age_band`, then against two broken versions, which your rows must catch.

Your code must pass these checks:

- Include rows for ages 34, 35, 54, 55 and None, both sides of each threshold and the missing value.
- Every row must pass for a correct age_band. Check the expected band for each age.
- Your rows must catch a < that became <=. Ages 35 and 55 do that when their expected bands are right.
- Add a row (None, None), so a missing age that lands in 55+ fails.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Age group for reporting. None stays None.
def age_band(age: int | None) -> str | None: ...

def test_age_band(
    age: int | None,
    expected: str | None,
) -> None: ...
```

Starter code:

```python
import pytest


def age_band(age: int | None) -> str | None:
    """Age group for reporting. None stays None."""
    if age is None:
        return None
    if age < 35:
        return "16-34"
    if age < 55:
        return "35-54"
    return "55+"

```

```
14
15 def test_age_band():
16     for age, expected in [(20, "16-34"), (40, "35-54"), (70, "55+")]:
17         assert age_band(age) == expected
```

Exercise 5. Architecture design (level 4)

Pick the tool for each testing need.

Options: @pytest.mark.parametrize table, Fixture with params, Hypothesis property (lesson B17-L2), A second parametrized test for the error cases.

1. The same rule checked on 12 input and output pairs
2. Every QA test must run against both the pandas and the Polars implementation
3. Normalised weights must sum to 1 for any non-empty list of positive weights
4. Five invalid inputs must each raise ValueError

Sources

- Default case IDs join the values (test_share[1-4-0.25]); stacked parametrize runs every combination; fixture params run each test once per value; a strict xfail row that passes reports [XPASS(strict)] and fails the run. (verified: Run on pytest 9.2.0 in this repository's scratch environment.)
- test.pytestmark holds the parametrize mark, with argnames in args[0] and rows (tuples or ParameterSet with .values) in args[1], in pytest 9.0.2 and 9.2.0. (verified: Run in the dev and Pyodide-pin environments. The E4 checker reads it.)

B19-L4 Testing pandas and NumPy code

Concept

`==` on DataFrames compares cell by cell and returns another DataFrame. It treats two missing values as different and ignores dtypes. Use the pandas testing helpers instead. In pandas 3.0, `pd.testing.assert_frame_equal(result, expected)` checks:

- values, with missing values in the same place counted as equal,
- dtypes, so a count that silently became float fails,
- column names and their order,
- the index.

Float columns are compared with a relative tolerance of 1e-5 by default, so `0.1 + 0.2` matches `0.3`. Integer columns must match exactly. `pd.testing.assert_series_equal` does the same for one column.

Two options matter most:

- `check_like=True` ignores the order of columns, and of rows with the same index labels.
- `check_dtype=False` ignores dtypes. Reach for it rarely: a dtype change is often the bug.

Filtering keeps the original index labels. After `df[df["w"] > 0]` drops row 1, the index is `[0, 2]`, and a comparison with a freshly built expected frame fails on the index. Decide what the function promises. If it promises a fresh index, it calls `.reset_index(drop=True)` and the test stays strict.

To compare rows whatever their order, sort both frames by their key columns and reset both indexes before comparing.

For NumPy arrays, use `np.testing.assert_allclose(actual, expected)`. Its defaults are `rtol=1e-7` and `atol=0`, and it treats NaN as equal to NaN. A relative tolerance never accepts any difference from an expected 0, so add `atol` when an expected value is 0.

Test that a function leaves its input alone. Take `before = df.copy()`, call the function, then compare `df` with `before`. Without `.copy()`, `before` is the same object and the comparison can never fail.

Why it matters

A refactor of a respondent-count function introduced a missing value, so pandas turned the integer `n` column into float. Every downstream check still compared with `==` and passed. The report template formatted `n` with `{:,}` and printed "1,204.0 respondents" on 40 pages. `assert_frame_equal` against an expected frame with an `int64` column fails on that change.

Worked example: weak code

```
def test_drop_zero_weights():
    df = pd.DataFrame({"id": ["R1", "R2", "R3"], "w": [1.0, 0.0, 2.0]})
    result = drop_zero_weights(df)
    assert (result["w"].values == [1.0, 2.0]).all()
```

1. **Line 4.** Checks one column's values only. A dropped id column, a dtype change or a mutated input all pass.

Worked example: strong code

```
 1 import pandas as pd
 2
 3
 4 def test_drop_zero_weights() -> None:
 5     df = pd.DataFrame({"id": ["R1", "R2", "R3"], "w": [1.0, 0.0, 2.0]})
 6     before = df.copy()
 7
 8     result = drop_zero_weights(df)
 9
10     expected = pd.DataFrame({"id": ["R1", "R3"], "w": [1.0, 2.0]})
11     pd.testing.assert_frame_equal(result, expected)
12     pd.testing.assert_frame_equal(df, before)
```

1. **Line 6.** A real copy, so the last line can detect a function that changes its input.
2. **Line 10.** The whole expected frame, written out. Its fresh index states that the function resets it.
3. **Line 11.** Checks every column, the dtypes and the index, with float tolerance.

Compare whole frames with assert_frame_equal against an expected frame you write out, and copy the input to prove the function leaves it alone.

Recap card

- **What assert_frame_equal checks** Values (missing values in the same place count as equal), dtypes, column names and order, and the index. Floats with rtol 1e-5.
- **Compare rows whatever their order** Sort both frames by the key columns, reset both indexes, then assert_frame_equal(..., check_like=True).
- **Why a filtered result fails on the index** Filtering keeps the old labels, such as [0, 2]. Reset the index in the function if it promises a fresh one.
- **assert_allclose and an expected 0** Defaults are rtol=1e-7, atol=0. A relative tolerance of 0 accepts nothing, so add atol.
- **Test that a function leaves its input alone** before = df.copy(), call the function, then assert_frame_equal(df, before).

Practice

Exercise 1. Predict the output (level 1)

What does this print? (pandas 3.0)

```
1 import numpy as np
2 import pandas as pd
3
4 a = pd.DataFrame({"share": [0.5, np.nan]})
```

```
b = pd.DataFrame({"share": [0.5, np.nan]})
print((a == b).all().all())
pd.testing.assert_frame_equal(a, b)
print("equal")
```

Exercise 2. Multiple choice (level 2)

`drop_zero_weights(df)` returns `df[df["w"] > 0]`. Its test fails with `DataFrame.index are different`, although the rows look right. The function should return a fresh index. What is the right fix?

- a) End the function with `.reset_index(drop=True)` and keep the test as it is.
- b) Add `check_dtype=False` to the assertion.
- c) Compare `result.values == expected.values` instead.
- d) Add `check_like=True` to the assertion.

Why?

- r1) Filtering keeps the old labels [0, 2]. The function promises a fresh index, so the function resets it, and the strict test protects that promise.
- r2) Indexes never matter in tests.

Exercise 3. Fill the gap (level 2)

Compare an array of shares with the expected values. One expected value is exactly 0, and the result there is `1e-15`, so a relative tolerance alone fails.

```
np.testing._____(1)(shares, [0.25, 0.75, 0.0], _____(2)=1e-12)
```

Exercise 4. Write the function (level 3)

Write `assert_same_rows(actual, expected, keys)` for tests whose result order does not matter. It raises `AssertionError` unless both frames hold the same rows, values and dtypes, whatever the order of rows and columns.

Sort both frames by the `keys` columns, reset both indexes, then compare with `pd.testing.assert_frame_equal(..., check_like=True)`. Do not change the frames passed in.

Your code must pass these checks:

- Rows and columns in a different order must pass. Sort by keys, reset the index and use check_like=True.
- A different value must raise AssertionError.
- A dtype change must raise. Do not pass check_dtype=False.
- A frame with an extra row must raise.

- Leave the frames passed in unchanged. sort_values and reset_index return new frames.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Assert two frames hold the same rows, ignoring row and column order.
def assert_same_rows(
    actual: pd.DataFrame,
    expected: pd.DataFrame,
    keys: list[str],
) -> None: ...
```

Starter code:

```python
import pandas as pd


def assert_same_rows(actual, expected, keys):
    assert (actual.values == expected.values).all()
```

Exercise 5. Spot the bug (level 4)

`add_share` changes its input, which the team rule forbids. This test was written to catch exactly that, yet it passes. Tap the line in the test that hides the problem, then fix the test. Leave `add_share` as it is.

```python
import pandas as pd


def add_share(df: pd.DataFrame) -> pd.DataFrame:
    df["share"] = df["w"] / df["w"].sum()
    return df


def test_add_share_keeps_input():
    df = pd.DataFrame({"w": [1.0, 3.0]})
    before = df
    add_share(df)
    pd.testing.assert_frame_equal(df, before)
```

Write down the line number of the bug. Then write the fix.

Sources

- assert_frame_equal: NaN in the same place is equal; float columns use relative tolerance 1e-5 (1.000005 passes, 1.00002 fails); integers compare exactly; column order, index and dtype differences fail; check_like=True ignores column and row order by label. (verified: Run on pandas 3.0.6 and 3.0.2 (Pyodide pin).)

- np.testing.assert_allclose defaults rtol=1e-07, atol=0, equal_nan=True; 1e-12 against 0.0 fails without atol. (verified: Run on NumPy 2.4.6.)

B19-L5 Testing statistical and random code

Concept

You rarely know the exact right answer for a statistic by heart. Three kinds of test still pin it down.

1. Compare with an independent oracle. On a small input, check your function against a trusted library that computes the same thing. Your Wilson interval should match statsmodels 0.14:

```
from statsmodels.stats.proportion import proportion_confint

expected = proportion_confint(12, 40, alpha=0.05, method="wilson")
np.testing.assert_allclose(wilson_interval(12, 40), expected, rtol=1e-6)
```

The tolerance covers small differences, such as a z value rounded to 1.959964.

2. Control the randomness. A function that draws random numbers takes a `np.random.Generator` as a parameter. The test passes in `np.random.default_rng(7)`, often from a fixture, so every run draws the same numbers. Assert properties and tolerances, not exact draws: a test that hardcodes `0.4123` tells a reader nothing about why that number is right.

3. Derive the tolerance from theory. A simulation estimate is itself random. The share of 2,000 simulations in which a 95% interval covers the true value has a standard error of `sqrt(0.95 * 0.05 / 2000)`, about 0.0049. Assert that the coverage lies within four standard errors, between 0.931 and 0.969. A correct method passes. Pick parameters where a broken method fails clearly: at p = 0.05 and n = 30, the Wald interval covers only about 79% of the time.

Test invariants too. A probability lies in [0, 1]. A lower bound is at most the estimate. An interval narrows as n grows.

Never assert something that fails by chance. A test that checks `p_value < 0.05` on random data fails about one run in twenty, whatever the code does, and teaches the team to ignore red builds.

Why it matters

A team's bootstrap test asserted that the 95% interval for a mean was exactly `(4.812, 5.391)`, copied from one run. It had no seed, so it failed

about once a week. The team added a retry. Months later a refactor resampled without replacement, which made every interval far too narrow. The retried test stayed green, because it now only checked that the function returned two numbers. A seeded coverage test with a theory-based tolerance would have failed at once.

Worked example: weak code

```
def test_bootstrap_ci():
    lo, hi = bootstrap_ci([4.0, 5.0, 6.0, 5.5], n_boot=1000)
    assert (round(lo, 3), round(hi, 3)) == (4.812, 5.391)
```

1. **Line 2.** No generator passed in, so every run draws different resamples.
2. **Line 3.** One run's output, copied in. It fails by chance and says nothing about why the numbers are right.

Worked example: strong code

```
import math

import numpy as np
import pytest


@pytest.fixture
def rng() -> np.random.Generator:
    return np.random.default_rng(7)


def test_wilson_matches_statsmodels() -> None:
    from statsmodels.stats.proportion import proportion_confint

    expected = proportion_confint(12, 40, alpha=0.05, method="wilson")
    np.testing.assert_allclose(wilson_interval(12, 40), expected, rtol=1e-6)


def test_wilson_coverage(rng: np.random.Generator) -> None:
    sims, p, n = 2000, 0.05, 30
    successes = rng.binomial(n, p, size=sims)
    covered = [lo <= p <= hi for lo, hi in (wilson_interval(int(x), n) for x in successes)]
    se = math.sqrt(0.95 * 0.05 / sims)
    assert abs(np.mean(covered) - 0.95) < 4 * se
```

1. **Line 9.** A seeded generator from a fixture: the same draws on every run.
2. **Line 15.** An independent oracle for the exact value on a small input.
3. **Line 20.** Parameters where a broken method, such as the Wald interval, fails clearly.

4. **Line 24.** The tolerance comes from the simulation's own standard error, not from a guess.

Check exact values against an oracle, seed every random draw, and size simulation tolerances from the standard error.

Recap card

- **Oracle test for a statistic** On a small input, compare with an independent library, such as statsmodels proportion_confint, using assert_allclose.
- **Make random code testable** Pass a np.random.Generator in. Tests pass np.random.default_rng(seed), often from a fixture.
- **Tolerance for a simulated coverage** Standard error sqrt(p(1 - p) / sims). Allow about four standard errors: 0.931 to 0.969 for 0.95 over 2,000 simulations.
- **An assertion never to write** p_value < 0.05 on random data. It fails by chance about one run in twenty.

Practice

Exercise 1. Multiple choice (level 1)

Which test for `bootstrap_ci(values, rng, n_boot)` is reliable and catches real bugs?

- a) Run it once without a seed and assert the exact interval printed by that run.
- b) Pass `np.random.default_rng(7)`, then assert that lower <= mean <= upper and that the interval is narrower for 400 values than for 40.
- c) Assert that a t-test on the bootstrap means gives p < 0.05.
- d) Run it ten times without a seed and pass if most runs look sensible.

Why?

- r1) The seed makes every run identical, and the assertions are properties any correct interval has.
- r2) Seeds make code faster.

Exercise 2. Predict the output (level 2)

What does this print?

```
1 import math
2
3 p, sims = 0.95, 2000
```

```
se = math.sqrt(p * (1 - p) / sims)
low, high = p - 4 * se, p + 4 * se
print(round(se, 4), round(low, 3), round(high, 3))
```

Exercise 3. Fill the gap (level 2)

Check your Wilson interval against statsmodels on one small input.

```
from statsmodels.stats.proportion import proportion_confint

expected = proportion_confint(12, 40, alpha=0.05, method="_____(1)")
np.testing._____(2)(wilson_interval(12, 40), expected, rtol=1e-6)
```

Exercise 4. Write the function (level 3)

Write `coverage(ci_fn, rng, p, n, sims) -> float`, the core of a coverage test. It draws `sims` binomial success counts with `rng.binomial(n, p, size=sims)`, calls `ci_fn(successes, n)` for each count (as a Python `int`), and returns the share of intervals `(lo, hi)` with `lo <= p <= hi`. Raise `ValueError` if `sims` is below 1. Use only the `rng` passed in.

Your code must pass these checks:

- For the Wilson interval at p = 0.05, n = 30 and 2,000 simulations, coverage must be close to 0.95. A result of 1.0 means you compared the interval with the estimate, not with the true p.
- For the Wald interval at p = 0.05 and n = 30, coverage is far below 0.95. Count an interval only when lo <= p <= hi.
- An interval of (0, 1) always covers p, so coverage must be 1.0.
- The same seed must give the same coverage. Draw only from the rng passed in.
- Call ci_fn once per simulation, with the count as a Python int.
- Raise ValueError when sims is below 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Share of simulated samples whose interval covers the true proportion.
def coverage(
    ci_fn: Callable[[int, int], tuple[float, float]],
    rng: np.random.Generator,
    p: float,
    n: int,
    sims: int,
) -> float: ...
```

Starter code:

```
import numpy as np

```

```
3
4 def coverage(ci_fn, rng, p, n, sims):
5     return 0.95
```

Exercise 5. Architecture design (level 4)

Plan the tests for `weighted_mean_ci(values, weights, rng)`, which returns a bootstrap interval for a weighted mean.

Options: Compare with np.average(values, weights=weights) on four rows, Compare with the value the function printed last week.

1. Is the point estimate right?
2. Make the bootstrap reproducible
3. Does the interval cover often enough?
4. Edge cases

Sources

- statsmodels proportion_confint(12, 40, alpha=0.05, method="wilson") matches a Wilson interval with z = 1.959964 within rtol 1e-6. (verified: Run on statsmodels 0.14.6 (Pyodide pin) and 0.15.0.)
- With np.random.default_rng(7), 2,000 simulations at p = 0.05, n = 30: Wilson coverage 0.9415, Wald coverage 0.7865. At p = 0.3, n = 50: Wilson 0.9605, Wald 0.938. (verified: Run on NumPy 2.4.6 in the dev and Pyodide-pin environments.)

B19-L6 Testing machine learning pipelines

Concept

You cannot assert that a model's AUC is exactly 0.873. The next scikit-learn release, a new seed or one more row changes it. Test what must hold for any correct pipeline instead. With scikit-learn 1.8 and a few hundred synthetic rows, each of these tests runs in under a second.

Contracts. `predict_proba` returns one row per input and one column per class. Every row sums to 1, and every value lies between 0 and 1.

No leakage. Fit the pipeline on the training split only, then check what it learned. A `StandardScaler` inside it must hold the training means: `pipe.named_steps["standardscaler"].mean_` equals `X_train.mean(axis=0)`. If it holds the mean of all the data, the test rows leaked into preprocessing (lesson B10-L2).

Determinism. Set `random_state` in every splitter and estimator. Two fits on the same data must give the same predictions. A split without `random_state` gives different rows every run, and the test becomes flaky.

Beats a baseline. `DummyClassifier(strategy="prior")` ignores the features, so its AUC is 0.5. On fixed data from `make_classification(random_state=0)`, assert that the model clearly beats it. A fixed high bar, such as AUC above 0.9, breaks on harmless changes. A margin over the baseline catches a pipeline that has stopped learning.

Behaviour. A model's prediction for a row must not depend on where the row sits. Shuffle the rows, predict, put the predictions back in the original order, and compare. A feature that raises risk must not lower the predicted risk when it alone increases, if the model promises that.

For your own transformers, scikit-learn's `parametrize_with_checks` from `sklearn.utils.estimator_checks` runs its standard API checks as pytest tests.

Why it matters

A churn pipeline's only test asserted `auc > 0.85` on a split without `random_state`. It failed one run in eight, and the team added a retry. Later a refactor moved the scaler outside the pipeline and fitted it on all rows. Offline AUC rose by 0.03 and the test stayed green. In production, AUC fell by 0.05. A one-line test comparing the fitted scaler's means with the training means fails on that refactor.

Worked example: weak code

```
def test_model():
    X_train, X_test, y_train, y_test = train_test_split(X, y)
    model = build_pipeline().fit(X_train, y_train)
    assert roc_auc_score(y_test, model.predict_proba(X_test)[:, 1]) > 0.85
```

1. **Line 2.** No random_state: different rows every run, so the test is flaky.
2. **Line 4.** One fixed bar. It cannot tell leakage from skill, and it fails on harmless changes.

Worked example: strong code

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.dummy import DummyClassifier
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split


def split():
```

```
    X, y = make_classification(n_samples=300, random_state=0)
    return train_test_split(X, y, test_size=0.3, random_state=0,
stratify=y)


def test_scaler_fitted_on_training_rows_only() -> None:
    X_train, _, y_train, _ = split()
    pipe = build_pipeline().fit(X_train, y_train)
    np.testing.assert_allclose(pipe.named_steps["standardscaler"].mean_,
X_train.mean(axis=0))


def test_probabilities_are_valid() -> None:
    X_train, X_test, y_train, _ = split()
    proba = build_pipeline().fit(X_train, y_train).predict_proba(X_test)
    assert proba.shape == (len(X_test), 2)
    np.testing.assert_allclose(proba.sum(axis=1), 1.0)


def test_beats_baseline() -> None:
    X_train, X_test, y_train, y_test = split()
    model = build_pipeline().fit(X_train, y_train)
    baseline = DummyClassifier(strategy="prior").fit(X_train, y_train)
    model_auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
    base_auc = roc_auc_score(y_test, baseline.predict_proba(X_test)[:, 1])
    assert model_auc > base_auc + 0.2
```

1. **Line 10.** Fixed synthetic data and a seeded split: the same rows every run.
2. **Line 16.** A leakage test: the scaler must have learned from training rows only.
3. **Line 22.** A contract: one row per input, one column per class.
4. **Line 32.** A margin over a baseline that ignores the features, not a fixed bar.

Test contracts, leakage, determinism and a margin over a baseline on fixed data, not one exact score.

Recap card

- **Leakage test for a pipeline** Fit on the training split, then assert pipe.named_steps["standardscaler"].mean_ equals X_train.mean(axis=0).
- **Contract tests for a classifier** predict_proba shape is (n_rows, n_classes), each row sums to 1, values lie in [0, 1].
- **Performance test that is not fragile** On fixed data from make_classification(random_state=0), assert the model beats DummyClassifier(strategy="prior") by a margin.
- **Behavioural test: row order** Shuffle with a seeded permutation and check each row's prediction is unchanged: compare with original[perm].

Practice

Exercise 1. Multiple choice (level 1)

Which test fails when the scaler has been fitted on all the data instead of the training rows?

- a) Fit on the training split, then assert the scaler's `mean_` equals `X_train.mean(axis=0)`.
- b) Assert test AUC is above 0.85.
- c) Assert `predict_proba` rows sum to 1.
- d) Assert the pipeline fits without raising.

Why?

- r1) It checks what the pipeline learned, which differs exactly when test rows leak in.
- r2) Means are faster to compute than AUC.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X_train = np.array([[1.0], [3.0], [5.0], [7.0]])
y_train = np.array([0, 0, 1, 1])
X_test = np.array([[100.0]])

pipe = make_pipeline(StandardScaler(), LogisticRegression()).fit(X_train,
    y_train)
pipe.predict(X_test)
print(pipe.named_steps["standardscaler"].mean_[0])
```

Exercise 3. Fill the gap (level 2)

Compare the model with a baseline that ignores the features.

```
baseline = DummyClassifier(strategy="_____(1)").fit(X_train, y_train)
base_auc = roc_auc_score(y_test, baseline._____(2)(X_test)[:, 1])
model_auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
assert model_auc > base_auc + 0.2
```

Exercise 4. Write the function (level 3)

Write `order_invariant(model, X, rng) -> bool`, a behavioural check. Draw a permutation with `rng.permutation(len(X))`, predict probabilities for the original rows and for the shuffled rows, and return `True` when each row gets

the same probabilities either way (within `np.allclose`). `X` is a NumPy array. Do not change it.

What you need to know.

`np.allclose(a, b)` returns `True` when every pair of values is close, with `rtol=1e-05` and `atol=1e-08` by default. Unlike `np.testing.assert_allclose`, it returns a bool instead of raising.

Indexing an array with an array of positions picks rows in that order and returns a new array:

```
scores = np.array([[10], [20], [30]])
scores[[2, 0, 1]]   # rows 30, 10, 20
```

Your code must pass these checks:

- A logistic regression scores each row on its own, so the check must return True.
- Compare each row's probabilities with the same row's shuffled result: align them with the permutation before comparing.
- Leave X unchanged. Index with the permutation instead of shuffling in place.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# True when each row's predicted probabilities do not depend on row order.
def order_invariant(
    model: object,
    X: np.ndarray,
    rng: np.random.Generator,
) -> bool: ...
```

Starter code:

```
import numpy as np


def order_invariant(model, X, rng):
    return True
```

Exercise 5. Spot the bug (level 4)

This test fails about one run in seven, with no code change. Tap the line that makes it flaky, then fix it.

```
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split

```

```
6
7  def test_model_beats_threshold():
8      X, y = make_classification(n_samples=300, random_state=0)
9      X_train, X_test, y_train, y_test = train_test_split(X, y,
       test_size=0.3)
10     model = RandomForestClassifier(n_estimators=50,
       random_state=0).fit(X_train, y_train)
11     auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
12     assert auc > 0.97
```

Write down the line number of the bug. Then write the fix.

Sources

- DummyClassifier with strategy prior or most_frequent gives AUC 0.5; a fitted StandardScaler in a pipeline holds the training means, unchanged by predict; train_test_split without random_state returns different rows on each call; parametrize_with_checks and check_estimator import from sklearn.utils.estimator_checks. (verified: Run on scikit-learn 1.8.0 (Pyodide pin) and 1.9.1.)
- The E5 test with an unseeded split fails (AUC <= 0.97) in 14.3% of 1,000 seeded splits; its AUC never fell to 0.9 in 1,000 splits. (verified: Run on scikit-learn 1.9.1 with make_classification(n_samples=300, random_state=0) and a 50-tree forest.)

B19-L7 Isolate I/O with fakes, monkeypatch and mocks

Concept

A unit test should not call a survey API, read a production table or depend on today's date. Lesson B2-L1 pushed side effects to the edges. Three tools handle what is left at those edges.

Pass the dependency in. A function that takes `get` or `now` as a parameter needs no tricks to test. The test passes a small fake:

```
1  class FakeResponse:
2      status_code = 200
3
4      def json(self) -> dict[str, int]:
5          return {"respondents": 812}
6
7
8  assert wave_count("W14", get=lambda url, timeout: FakeResponse()) == 812
```

`monkeypatch`, a built-in fixture, changes something for one test and undoes it afterwards. `monkeypatch.setenv("SURVEY_API_TOKEN", "test-token")` sets an

environment variable. `monkeypatch.delenv("SURVEY_API_TOKEN", raising=False)` removes it. `monkeypatch.setattr(module, "name", value)` replaces an attribute.

`unittest.mock` from the standard library builds stand-ins that record how they were called:

- `Mock(return_value=x)` returns `x` on every call.
- `Mock(side_effect=[a, ConnectionError("down")])` returns `a`, then raises on the next call.
- `fake.assert_called_once_with(url, timeout=30)` checks the arguments.
- `fake.call_count` counts calls, including calls that raised.

`patch("module.name")` swaps a name for the length of a `with` block. Patch the name **where the code looks it up**. If `reports.py` does `from requests import get`, patch `reports.get`. Patching `requests.get` leaves `reports.get` pointing at the real function, and the test makes a real call.

Use mocks sparingly. A mock checks how your code talks to the outside world, not whether its result is right. Mock the boundary, such as HTTP, the clock or a database, and test the logic with real objects. Never mock pandas.

Why it matters

A test of a weekly loader patched `requests.get`, but the loader imported `get` by name. Every CI run quietly called the supplier's live API with a test token. The supplier rate-limited the account during a delivery week, and the real nightly load failed. Patching `loader.get` instead, or passing `get` in as a parameter, would have kept the tests offline.

Worked example: weak code

```
 1 import requests
 2
 3
 4 def wave_count(wave_id: str) -> int:
 1     response = requests.get(f"https://survey.example/api/waves/
       {wave_id}", timeout=30)
 6     return response.json()["respondents"]
 7
 8
 9 def test_wave_count():
 2     assert wave_count("W14") > 0
```

1. **Line 5.** The network call is buried inside the logic, so the test cannot avoid it.
2. **Line 10.** Calls the live API on every run, and asserts almost nothing.

Worked example: strong code

```
from collections.abc import Callable
from typing import Any

import pytest

API = "https://survey.example/api/waves"


def wave_count(wave_id: str, get: Callable[..., Any]) -> int:
    response = get(f"{API}/{wave_id}", timeout=30)
    if response.status_code != 200:
        raise RuntimeError(f"wave API returned {response.status_code}")
    return int(response.json()["respondents"])


class FakeResponse:
    def __init__(self, status_code: int, payload: dict[str, int]) -> None:
        self.status_code = status_code
        self._payload = payload

    def json(self) -> dict[str, int]:
        return self._payload


def test_wave_count_reads_respondents() -> None:
    calls = []

    def fake_get(url: str, timeout: int) -> FakeResponse:
        calls.append((url, timeout))
        return FakeResponse(200, {"respondents": 812})

    assert wave_count("W14", fake_get) == 812
    assert calls == [(f"{API}/W14", 30)]


def test_wave_count_raises_on_error_status() -> None:
    with pytest.raises(RuntimeError, match="503"):
        wave_count("W14", lambda url, timeout: FakeResponse(503, {}))
```

1. **Line 9.** The HTTP function is a parameter. Production passes requests.get, tests pass a fake.
2. **Line 16.** A tiny fake with only what the code uses: status_code and json().
3. **Line 33.** Checks the request too: the right URL and a timeout.
4. **Line 37.** The error path is tested without a real outage.

Pass I/O in as a parameter where you can, use monkeypatch for settings, and patch a name where the code looks it up.

Recap card

- **Easiest way to test code that does I/O** Pass the dependency in as a parameter, such as get or now. Tests pass a small fake.
- **Change an environment variable for one test** monkeypatch.setenv(name, value) or monkeypatch.delenv(name, raising=False). pytest restores it afterwards.
- **Mock that fails on the second call** Mock(side_effect=[result, ConnectionError("down")]). call_count counts both calls.
- **Where to patch** Where the code looks the name up. After from requests import get in reports.py, patch reports.get.

Practice

Exercise 1. Multiple choice (level 1)

`fetch.py` starts with `from requests import get` and calls `get(url, timeout=30)`. Which patch keeps a test of `fetch.load()` offline?

- a) `patch("requests.get")`
- b) `patch("fetch.get")`
- c) `patch("get")`
- d) `patch("fetch.requests.get")`

Why?

- r1) fetch.py looks up the name get in its own module, so that is the name to replace.
- r2) Patching a module's name is faster than patching requests.

Exercise 2. Predict the output (level 2)

What does this print?

```
from unittest.mock import Mock

api = Mock(side_effect=[{"n": 10}, ConnectionError("down")])
print(api("W1")["n"])
try:
    api("W2")
except ConnectionError as err:
    print("error:", err)
print(api.call_count)
```

Exercise 3. Fill the gap (level 2)

Test both paths of `load_token()`, which reads `SURVEY_API_TOKEN` and raises `RuntimeError` when it is missing. Leave the real environment untouched.

```python
def test_missing_token_raises(monkeypatch):
    monkeypatch._____(1)("SURVEY_API_TOKEN", raising=False)
    with pytest.raises(RuntimeError, match="SURVEY_API_TOKEN"):
        load_token()


def test_token_is_read(monkeypatch):
    monkeypatch._____(2)("SURVEY_API_TOKEN", "test-token")
    assert load_token() == "test-token"
```

Exercise 4. Refactor (level 3)

Make `wave_count` testable without a network. Its new signature is `wave_count(wave_id: str, get: Callable[..., Any]) -> int`.

- Call `get(f"https://survey.example/api/waves/{wave_id}", timeout=30)`.
- Raise `RuntimeError` naming the status code when `response.status_code` is not 200.
- Return `int(response.json()["respondents"])`.
- Remove the `requests` import. The caller passes `requests.get` in production.

Your code must pass these checks:

- Call get once with the wave URL and timeout=30, and return the respondents count as an int.
- Raise RuntimeError naming the status code when it is not 200.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
API = ...

# Number of respondents in one wave, from the survey API.
def wave_count(wave_id: str, get: Callable[..., Any]) -> int: ...
```

Starter code:

```python
import requests


def wave_count(wave_id):
    response = requests.get(f"https://survey.example/api/waves/
    {wave_id}", timeout=30)
    return response.json()["respondents"]
```

Exercise 5. Spot the bug (level 4)

This test calls the live API on every run. Tap the line that lets the real call through, then fix it.

```python
# reports.py contains:
#     from requests import get
#
#     def wave_count(wave_id):
#         url = f"https://survey.example/api/waves/{wave_id}"
#         return get(url, timeout=30).json()["respondents"]

from unittest.mock import patch

import reports


def test_wave_count_reads_respondents():
    with patch("requests.get") as fake_get:
        fake_get.return_value.json.return_value = {"respondents": 812}
        assert reports.wave_count("W14") == 812
```

Write down the line number of the bug. Then write the fix.

Sources

- patch on the original module leaves a name imported with from ... import pointing at the real function; patching the importing module's name works; assert_called_once_with checks arguments. (verified: Run on Python 3.11 with pytest 9.2.0, using a stand-in module for requests.)
- Mock(side_effect=[value, exception]) returns then raises, and call_count counts the raising call; monkeypatch.delenv(raising=False) and setenv are undone after the test. (verified: Run on Python 3.11 and 3.13.)

B19-L8 Coverage, markers and a test suite you can trust in CI

Concept

Coverage tells you which code your tests ran. It does not tell you whether they checked anything. A test whose only assertion is `assert result` reaches every line and catches almost nothing (lesson B17-L1). Use coverage to find code no test reaches, not as proof of quality.

With pytest-cov 7.1:

```
pytest --cov=survey_stats --cov-branch --cov-report=term-missing --cov-fail-under=90
```

`--cov-branch` adds **branch coverage**. Line coverage counts a line once it runs. Branch coverage also asks whether each `if` went both ways. A function with `if cap is not None:` and a single test that passes a cap runs

every line, 100% line coverage, but never takes the path where `cap` is `None`. The report lists that missed jump as `2->4` in the Missing column, and the file drops to 83%.

`--cov-fail-under=90` makes pytest exit with an error when total coverage falls below 90%. Agree a floor with the team and raise it over time. Do not chase 100% with tests that assert nothing.

Markers label tests. Register each one in `pyproject.toml`, and turn on `--strict-markers`, so a typo such as `@pytest.mark.slwo` stops the run instead of creating a new, unused marker:

```
[tool.pytest.ini_options]
addopts = "--strict-markers"
markers = ["slow: takes more than a few seconds"]
```

Run `pytest -m "not slow"` while you work, and the full suite in CI. pytest 9 also accepts `strict = true`, which turns on strict markers along with other strict checks.

A **flaky** test passes and fails without a code change. Find the cause: a missing seed, shared state, the clock or the network. Never add a retry and move on.

Coverage misses weak assertions. **Mutation testing** finds them: mutmut 3.8 changes your code in small ways, such as `<` to `<=`, and reports every change that no test catches. It automates the habit of breaking the code once.

Why it matters

A pricing module had 96% line coverage, and the team was proud of it. Branch coverage was 71%. Every `else` path that handled a missing discount code was untested, including one that applied a 100% discount. It shipped and cost a client three days of free orders. `--cov-branch` with a 90% floor would have blocked the merge and pointed at the exact branches.

Worked example: weak code

```
# pyproject.toml has no marker settings
import pytest


@pytest.mark.slwo
def test_coverage_simulation():
    ...


def test_capped():
```

```
❷     assert capped(5, cap=3)
```

1. **Line 5.** A typo. Without --strict-markers it only warns, so `-m "not slow"` still runs this slow test.
2. **Line 11.** Runs every line of capped, so line coverage says 100%. It checks nothing, and the cap=None path never runs.

Worked example: strong code

```
 1 # pyproject.toml:
❶ #   addopts = "--strict-markers"
 3 #   markers = ["slow: takes more than a few seconds"]
 4 import pytest
 5
 6
❷ @pytest.mark.slow
 8 def test_coverage_simulation(rng):
 9     ...
10
11
12 @pytest.mark.parametrize(
13     ("weight", "cap", "expected"),
❸     [(5, 3, 3), (2, 3, 2), (5, None, 5)],
15 )
16 def test_capped(weight, cap, expected):
17     assert capped(weight, cap=cap) == expected
```

1. **Line 2.** An unregistered marker now stops the run.
2. **Line 7.** A registered marker. `pytest -m "not slow"` skips it locally, and CI runs it.
3. **Line 14.** Rows for both sides of the if, so branch coverage reaches 100% and each row asserts a value.

Measure branch coverage and set a floor, register markers strictly, fix flaky tests at the cause, and remember that coverage says what ran, not what was checked.

Recap card

- **Line coverage against branch coverage** Line: did each line run? Branch (--cov-branch): did each if go both ways? A missed jump shows as 2->4 in Missing.
- **Fail CI when coverage drops** pytest --cov=pkg --cov-branch --cov-fail-under=90. Start near today's level and raise it.
- **Catch marker typos** Register markers under markers = [...] and add --strict-markers to addopts. pytest 9 also offers strict = true.

- **What coverage cannot tell you** Whether tests check anything. Mutation testing (mutmut 3.8) changes the code and reports changes no test catches.
- **A flaky test** Fix the cause: seed, shared state, clock or network. A retry hides it.

Practice

Exercise 1. Multiple choice (level 1)

A module has 100% line coverage, yet a bug in the path where `cap` is `None` shipped. Which setting would have pointed at the untested path?

- a) `--cov-branch`
- b) `--cov-fail-under=100` on line coverage
- c) `-m "not slow"`
- d) `--strict-markers`

Why?

- r1) Branch coverage reports each if that never went one of its two ways, even when every line ran.
- r2) Branch coverage makes the tests run faster.

Exercise 2. Predict the output (level 2)

`survey/weights.py` holds this function, and the only test is `assert capped(5, cap=3) == 3`. pytest-cov reports the file's coverage without `--cov-branch` and with it. What are the two percentages? Answer as two numbers, such as `90 75`.

```
def capped(w, cap=None):
    if cap is not None:
        w = min(w, cap)
    return w
```

Exercise 3. Fill the gap (level 2)

In `pyproject.toml`, register a `slow` marker and make any unregistered marker stop the run.

```
[tool.pytest.ini_options]
addopts = "_____(1)"
markers = ["_____(2): takes more than a few seconds"]
```

Exercise 4. Write the function (level 3)

`risk_band(score)` is already imported in your test file:

```python
def risk_band(score: float | None) -> str:
    if score is None:
        return "unknown"
    if score >= 0.8:
        return "high"
    if score >= 0.5:
        return "medium"
    return "low"
```

Write tests, as functions whose names start with `test_`, that run every branch **and** catch an off-by-one change. The checker measures which lines your tests run, then runs them against two changed versions: `>= 0.8` becomes `> 0.8`, and `>= 0.5` becomes `> 0.5`. Your tests must fail for both.

Your code must pass these checks:

- Your tests must pass for the correct risk_band. Check each expected band.
- Run every branch: include a missing score, a high, a medium and a low score.
- Coverage is not enough: add a case exactly at 0.8, which must be high.
- Add a case exactly at 0.5, which must be medium.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
def test_risk_band(
    score: float | None,
    expected: str,
) -> None: ...
```

Starter code:

```python
def test_risk_band():
    assert risk_band(0.9) == "high"
    assert risk_band(0.1) == "low"
```

Exercise 5. Architecture design (level 4)

Set up the team's test runs.

Options: pytest with --cov-branch and --cov-fail-under at the agreed floor, all tests including slow, No tests, to keep pull requests quick.

1. Every pull request
2. While you work on your laptop
3. A test fails one run in ten with no code change
4. The coverage floor

Practical: Build a complete test suite for a survey package

Platform: Laptop. Work in `labs/testing/` in this repository. It holds a small package, `survey_stats`, with age bands, weight helpers, a Wilson interval and an API client. You need the `dev` dependency group, which includes pytest 9, pytest-cov and statsmodels. Each step uses one lesson of module B19.

1. From the repository root, run `pip install -e . --group dev`, then `cd labs/testing` and `pytest`. One example test passes. Read every file in `src/survey_stats/`.
2. In `tests/test_bands.py`, write a parametrized test for `age_band` with named rows on both sides of 35 and 55, and a missing age (lesson B19-L3).
3. In `tests/conftest.py`, write a `make_respondents` factory fixture. Use it in `tests/test_weights.py` to test `cap_weights` with `assert_frame_equal`, including a missing weight and a check that the input is unchanged (lessons B19-L2 and B19-L4).
4. Test both errors of `normalise` and of `cap_weights` with `pytest.raises` and `match` (lesson B19-L1).
5. In `tests/test_intervals.py`, compare `wilson_interval` with statsmodels, then write a seeded coverage simulation with a tolerance of four standard errors. Mark it `@pytest.mark.slow` (lessons B19-L5 and B19-L8).
6. In `tests/test_api.py`, test `wave_count` with a fake `get`: one test for a 200 response that also checks the URL and timeout, one for a 503 (lesson B19-L7).
7. Run `pytest --cov=survey_stats --cov-branch --cov-report=term-missing`. Add tests until the Missing column is empty. Then run `pytest -m "not slow"` and check the simulation is deselected.
8. Break each function once, for example change `<` to `<=` in `age_band`. Confirm a test fails each time, then undo the change. Finally run `pytest reference_tests` and compare the reference suite with yours.

Sources

- With one test passing a cap, pytest-cov reports 100% line coverage and 83% with --cov-branch, listing the missed jump 2->4; --cov-fail-under=90 exits 1 with 'Coverage failure'. (verified: Run on pytest 9.2.0, pytest-cov 7.1.0, coverage 7.17.0.)
- An unregistered marker warns (PytestUnknownMarkWarning) without strict mode, and stops collection with --strict-markers or strict = true; -m

"not slow" deselects marked tests. (verified: Run on pytest 9.0.2 and 9.2.0.)

- mutmut 3.8.1 is the current release on PyPI. (search-only: pip index versions mutmut, October 2026.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B19-L1 How pytest finds, runs and reports your tests

B19-L1 exercise 1

Hints. (1) Check the file name and the function name separately. (2) Files: `test_*.py` or `*_test.py`. Functions: `test*`. (3) Only one option matches both.

Answer. c) `def test_weights():` in `tests/weights_test.py` Because: r1) Both the file name (`*_test.py`) and the function name (`test*`) match the default patterns.

Collection needs a matching file name and a matching function name. A class needs a `Test` prefix and no `__init__`.

Common wrong answers.

- `a`: The file matches, but the function name must start with `test`.
- `b`: The function matches, but `weight_checks.py` matches neither `test_*.py` nor `*_test.py`.
- `d`: Test classes must start with `Test`. `WeightTests` does not.
- `c|r2`: Right choice, wrong reason. pytest collects only names that match its patterns.

B19-L1 exercise 2

Hints. (1) `match` is searched in the message with a regular expression. (2) Does `total must be >= 0, got -5` contain `got -` and a digit? (3) Does it contain `must be > 0`?

Answer.

```
1 first ok
2 pattern did not match
```

Both blocks raise a ValueError. Only the first message matches its pattern. A failed match is reported as an AssertionError.

Common wrong answers.

- `first ok`: The second block raises a ValueError, but its message does not contain `must be > 0`. A failed match raises AssertionError, which the `except` catches.

- `pattern did not match`: `match` is a regular expression searched in the message. `got -\d` finds `got -5`, so the first block passes.

B19-L1 exercise 3

Hints. (1) `-k` takes an expression matched against test names. (2) The stop-early flag is one letter. (3) `-k zero -x`.

Answer.

```
pytest -k zero -x
```

`-k zero` selects tests whose names contain `zero`. `-x` stops the run at the first failure.

Common wrong answers.

- g1 = `zero`; g2 = `s`: `-s` shows print output. The flag that stops at the first failure is `-x`.
- g1 = `zero`; g2 = `q`: `-q` makes the output shorter. Stopping early is `-x`.

B19-L1 exercise 4

Hints. (1) A try/except that ignores the 'no error' case passes for broken code. (2) `with pytest.raises(ValueError, match="..."):` around the call. (3) Use `negative weight` and `total weight is 0` as the patterns.

Answer.

```
import pytest


def normalise(weights: list[float]) -> list[float]:
    """Scale weights so they sum to 1."""
    for i, w in enumerate(weights):
        if w < 0:
            raise ValueError(f"negative weight at index {i}")
    total = sum(weights)
    if total == 0:
        raise ValueError("total weight is 0")
    return [w / total for w in weights]


def test_negative_weight_raises() -> None:
    with pytest.raises(ValueError, match="negative weight"):
        normalise([2.0, -1.0])


def test_zero_total_raises() -> None:
    with pytest.raises(ValueError, match="total weight is 0"):
        normalise([0.0, 0.0])
```

`pytest.raises` fails when nothing is raised and lets other exception types through. `match` rules out a ValueError raised for the wrong reason.

B19-L1 exercise 5

Hints. (1) Start from what CI tells you. (2) Reproduce and read before you change anything. (3) Widen the run step by step: one test, last failures, everything.

Answer.

1. Copy the failing test's node ID from the CI log.
2. Run that one test locally: `pytest tests/test_share.py::test_zero_total`.
3. Read the assertion output: the values on both sides and the call that produced them.
4. Change the code until that test passes.
5. Run `pytest --lf` to rerun every test that failed last time.
6. Run the whole suite, then push.

Reproduce the single failure, read pytest's output, fix, then widen the run: last failures first, the whole suite last.

Common wrong answers.

- Read the failure before you change code. The assertion output often names the bug.
- `--lf` is the quick check before the full run, not after it.

B19-L2 Fixtures, shared setup without shared state

B19-L2 exercise 1

Hints. (1) What happens to a shared object when one test changes it? (2) Which scope builds a new object for every test? (3) The default.

Answer. a) The default, function scope Because: r1) Each test gets its own fresh frame, so one test's changes cannot reach another.

Function scope gives every test its own object. Wider scopes are for expensive objects that no test changes.

Common wrong answers.

- `b`: Module scope shares one frame across the file. A change in one test leaks into the next.
- `c`: Session scope shares one frame across the whole run. Save it for expensive, read-only objects.

- `d`: A module-level variable is shared state, the problem fixtures solve.
- `a|r2`: Right scope, wrong reason. A three-row frame is cheap either way. Isolation is the point.

B19-L2 exercise 2

Hints. (1) The fixture has function scope. (2) Code after `yield` runs when that test finishes. (3) Setup, test, teardown, then the same again.

Answer.

1. setup (first time)
2. test_sum
3. teardown (first time)
4. setup (second time)
5. test_len
6. teardown (second time)

For each test, pytest runs the code before `yield`, runs the test, then runs the code after `yield`.

Common wrong answers.

- A function-scoped fixture is torn down after each test and built again for the next.
- pytest builds a function-scoped fixture right before each test, not all at the start.

B19-L2 exercise 3

Hints. (1) The decorator that makes a fixture. (2) One per file is one per module. (3) Hand the value over and come back later for teardown.

Answer.

```
1 @pytest.fixture(scope="module")
2 def con():
3     connection = duckdb.connect()
4     yield connection
5     connection.close()
```

`@pytest.fixture(scope="module")` builds one connection per file. `yield` gives it to the tests, and the line after it runs once they finish.

Common wrong answers.

- g1 = `fixture`; g2 = `session`; g3 = `yield`: Session scope shares the connection across every file in the run. The task asks for one per file: module scope.

- g1 = `fixture`; g2 = `module`; g3 = `return`: `return` ends the function, so `connection.close()` never runs. `yield` hands over the value and resumes for teardown.

B19-L2 exercise 4

Hints. (1) The fixture defines an inner function `make` and returns it, without calling it. (2) Inside `make`, build the IDs with `f"R{i}"` and default `in_scope` to `[True] * len(weights)`. (3) In the test: `df = make_respondents([1.0, 3.0], in_scope=[True, False])`, then assert 1.0.

Answer.

```python
import pandas as pd
import pytest


def in_scope_weight(df: pd.DataFrame) -> float:
    """Total final_weight of in-scope rows."""
    return float(df.loc[df["in_scope"], "final_weight"].sum())


@pytest.fixture
def make_respondents():
    """Factory for small respondent frames with sensible defaults."""

    def make(weights: list[float], in_scope: list[bool] | None = None) -> pd.DataFrame:
        ids = [f"R{i}" for i in range(1, len(weights) + 1)]
        flags = in_scope if in_scope is not None else [True] * len(weights)
        return pd.DataFrame(
            {
                "respondent_id": ids,
                "final_weight": [float(w) for w in weights],
                "in_scope": flags,
            }
        )

    return make


def test_in_scope_weight_skips_out_of_scope(make_respondents) -> None:
    df = make_respondents([1.0, 3.0], in_scope=[True, False])
    assert in_scope_weight(df) == 1.0
```

The factory builds a new frame on every call, with defaults, so each test states only the values it cares about and no test can change another's data.

B19-L2 exercise 5

Hints. (1) Which test sees different data depending on what ran before? (2) One line shares the frame. Another changes it. (3) The scope argument and the in-place assignment.

Answer. The bug is on line 5, 11.

`scope="module"` builds one frame for the file, and `test_cap_weights` changes it in place. Use function scope, or have the code under test return a new frame.

Common wrong answers.

- Line 5: Module scope shares the frame, which is only a problem because a test changes it. Find the change too.
- Line 11: That line changes the frame. It leaks into the next test only because the fixture is shared. Find what shares it.
- Line 16: This assertion is correct for the fixture's data. It fails because of what happened to the frame earlier.

B19-L3 Parametrize, one test and many readable cases

B19-L3 exercise 1

Hints. (1) Two decorators on one test combine every value with every value. (2) A fixture with params runs each test that uses it once per value. (3) 6 + 2.

Answer.

```
1 8
```

`test_grid` runs for 2 x 3 = 6 combinations and `test_backend` once per param: 8 tests.

Common wrong answers.

- `7`: Stacked parametrize decorators multiply: 2 parts times 3 totals is 6, plus 2 backends.
- `3`: Each row and each fixture param becomes its own test. Count the combinations.
- `5`: Two stacked decorators give every combination, 2 x 3 = 6, not 2 + 3.

B19-L3 exercise 2

Hints. (1) The row now passes. Is that expected? (2) What does strict change? (3) An unexpected pass.

Answer. b) The row is reported as XPASS(strict) and the run fails. Because: r1) strict=True turns an unexpected pass into a failure, so the stale mark gets removed.

With strict=True, a row expected to fail that passes is reported as XPASS(strict) and fails the run, so you remove the mark.

Common wrong answers.

- `a`: That is what happens without strict=True: an xpass is reported, and the stale mark stays forever.
- `c`: xfail runs the row. skip does not.
- `d`: pytest never edits your code. Failing the run is how it tells you.
- `b|r2`: Right outcome, wrong reason. An xfail row can pass. strict=True makes that pass fail the run.

B19-L3 exercise 3

Hints. (1) The next row shows the pattern. (2) One row with options. (3) `pytest.param(..., id=...)`.

Answer.

```python
@pytest.mark.parametrize(
    ("age", "expected"),
    [
        pytest.param(34, "16-34", id="just-below-35"),
        pytest.param(35, "35-54", id="at-35"),
    ],
)
def test_age_band(age, expected):
    assert age_band(age) == expected
```

`pytest.param` wraps one row so it can carry an `id` and marks.

Common wrong answers.

- g1 = `param`; g2 = `name`: The keyword is `id`, as in the next row.
- g1 = `mark`; g2 = `id`: `pytest.mark` holds markers. A single row with options is `pytest.param`.

B19-L3 exercise 4

Hints. (1) One row per age, with its expected band: `(34, "16-34")`. (2) At 35 the band is already 35-54. At 55 it is already 55+. (3) Add `(None, None)` and keep a single assertion in the test body.

Answer.

```python
import pytest

```

```python

def age_band(age: int | None) -> str | None:
    """Age group for reporting. None stays None."""
    if age is None:
        return None
    if age < 35:
        return "16-34"
    if age < 55:
        return "35-54"
    return "55+"


@pytest.mark.parametrize(
    ("age", "expected"),
    [
        pytest.param(34, "16-34", id="below-35"),
        pytest.param(35, "35-54", id="at-35"),
        pytest.param(54, "35-54", id="below-55"),
        pytest.param(55, "55+", id="at-55"),
        pytest.param(None, None, id="missing"),
    ],
)
def test_age_band(age: int | None, expected: str | None) -> None:
    assert age_band(age) == expected
```

Rows on both sides of each threshold catch an off-by-one change, and the missing-value row catches None falling into the last band.

B19-L3 exercise 5

Hints. (1) Known pairs, or any input? (2) Which tool applies to every test that uses it? (3) Keep raising and non-raising cases apart.

Answer.

- A table of known cases: parametrize.
- The same tests against several implementations: a fixture with params, used by every test.
- A rule for all inputs: a Hypothesis property.
- Invalid inputs: their own parametrized test with pytest.raises, not an if inside the value test.

Tables for known cases, fixture params for implementations, properties for general rules, and a separate table for errors.

Common wrong answers.

- backends = `parametrize`: Parametrizing each test by hand repeats the backend list in every test. A fixture with params applies it to all of them.

- errors = `parametrize`: Mixing raising and non-raising rows in one table forces an if into the test body. Give errors their own test.
- property = `parametrize`: A table checks the cases you thought of. A property checks generated cases, including the ones you did not.

B19-L4 Testing pandas and NumPy code

B19-L4 exercise 1

Hints. (1) Is NaN == NaN? (2) How does assert_frame_equal treat missing values in the same place? (3) One check says different, the other says equal.

Answer.

```
1 False
2 equal
```

`==` follows NaN != NaN. `assert_frame_equal` treats missing values in matching positions as equal, which is what a test needs.

Common wrong answers.

- `True equal`: NaN is never equal to NaN under `==`, so the second cell is False and so is `.all().all()`.
- `False`: `assert_frame_equal` counts missing values in the same place as equal, so it does not raise and `equal` prints.

B19-L4 exercise 2

Hints. (1) What index does a filter keep? (2) Which side should change: the test or the function? (3) The function promises a fresh index.

Answer. a) End the function with `.reset_index(drop=True)` and keep the test as it is. Because: r1) Filtering keeps the old labels [0, 2]. The function promises a fresh index, so the function resets it, and the strict test protects that promise.

Filtering keeps the original labels. When the contract says the index is fresh, the function resets it and the test stays strict.

Common wrong answers.

- `b`: The dtypes match. The failure is about the index.
- `c`: That hides the index, the dtypes and the column names, so later bugs pass too.
- `d`: check_like reorders by label. It cannot turn labels [0, 2] into [0, 1].

- `a|r2`: Right fix, wrong reason. The index matters: a caller that uses .loc[1] gets an error with labels [0, 2].

B19-L4 exercise 3

Hints. (1) The NumPy helper that compares with a tolerance. (2) Relative tolerance times 0 is 0. (3) An absolute tolerance.

Answer.

```
np.testing.assert_allclose(shares, [0.25, 0.75, 0.0], atol=1e-12)
```

`assert_allclose` passes when `|actual - expected| <= atol + rtol * |expected|`. For an expected 0, only `atol` helps.

Common wrong answers.

- g1 = `assert_allclose`; g2 = `rtol`: A relative tolerance is a share of the expected value. A share of 0 is 0, so 1e-15 still fails. Use an absolute tolerance.
- g1 = `assert_array_equal`; g2 = `atol`: assert_array_equal checks exact equality and has no tolerance.

B19-L4 exercise 4

Hints. (1) `actual.sort_values(keys)` returns a new, sorted frame. (2) After sorting, `.reset_index(drop=True)` gives both frames the index 0, 1, 2. (3) Finish with `pd.testing.assert_frame_equal(left, right, check_like=True)`.

Answer.

```
import pandas as pd


def assert_same_rows(actual: pd.DataFrame, expected: pd.DataFrame, keys: list[str]) -> None:
    """Assert two frames hold the same rows, ignoring row and column order.

    Args:
        actual: Frame produced by the code under test.
        expected: Frame the test expects.
        keys: Columns that identify a row, used to sort both frames.

    Raises:
        AssertionError: If rows, values or dtypes differ.
    """
    left = actual.sort_values(keys).reset_index(drop=True)
    right = expected.sort_values(keys).reset_index(drop=True)
    pd.testing.assert_frame_equal(left, right, check_like=True)
```

Sorting by the keys and resetting the index makes row order irrelevant. `check_like=True` makes column order irrelevant. Everything else stays strict.

B19-L4 exercise 5

Hints. (1) What object does `before` refer to? (2) Assignment copies a reference, not the data. (3) `before = df.copy()`.

Answer. The bug is on line 11.

Fixed version:

```
 1 import pandas as pd
 2
 3
 4 def add_share(df: pd.DataFrame) -> pd.DataFrame:
 5     df["share"] = df["w"] / df["w"].sum()
 6     return df
 7
 8
 9 def test_add_share_keeps_input():
10     df = pd.DataFrame({"w": [1.0, 3.0]})
11     before = df.copy()
12     add_share(df)
13     pd.testing.assert_frame_equal(df, before)
```

`before = df` makes a second name for the same frame, so the comparison checks the frame against itself. `df.copy()` keeps the original values.

Common wrong answers.

- Line 5: That line does change the input, but the task is the test: why does it not notice?
- Line 13: The comparison is fine. It compares df with itself, because of how `before` was made.

B19-L5 Testing statistical and random code

B19-L5 exercise 1

Hints. (1) Will the test give the same result every run? (2) Does it assert something a correct interval must satisfy? (3) Seed plus invariants.

Answer. b) Pass `np.random.default_rng(7)`, then assert that lower <= mean <= upper and that the interval is narrower for 400 values than for 40. Because: r1) The seed makes every run identical, and the assertions are properties any correct interval has.

A seeded generator removes chance from the run. Invariants such as ordering and narrowing hold for every correct implementation.

Common wrong answers.

- `a`: Without a seed the draws change every run, so the copied interval fails by chance.
- `c`: A p-value on random data falls below 0.05 about one time in twenty, whatever the code does.
- `d`: Repeating an unseeded test only hides how often it fails. 'Looks sensible' is not an assertion.
- `b|r2`: Right test, wrong reason. A seed buys reproducibility, not speed.

B19-L5 exercise 2

Hints. (1) 0.95 x 0.05 = 0.0475. (2) 0.0475 / 2000 = 0.00002375. Take the square root. (3) Four times 0.0049 is about 0.0195.

Answer.

```
0.0049 0.931 0.969
```

The standard error of a share from 2,000 simulations is about 0.0049, so four standard errors give the band 0.931 to 0.969.

Common wrong answers.

- `0.0049 0.94 0.96`: The band is four standard errors wide on each side: 4 x 0.0049 is about 0.0195.
- `0.0024 0.94 0.96`: 0.95 x 0.05 / 2000 is 0.00002375, and its square root is about 0.0049.

B19-L5 exercise 3

Hints. (1) Name the same method as your function. (2) Compare with a tolerance. (3) `wilson` and `assert_allclose`.

Answer.

```
from statsmodels.stats.proportion import proportion_confint

expected = proportion_confint(12, 40, alpha=0.05, method="wilson")
np.testing.assert_allclose(wilson_interval(12, 40), expected, rtol=1e-6)
```

An independent library computing the same interval is an oracle. `assert_allclose` with a small `rtol` absorbs rounding differences.

Common wrong answers.

- g1 = `normal`; g2 = `assert_allclose`: `method="normal"` is the Wald interval. Compare like with like: `wilson`.
- g1 = `wilson`; g2 = `assert_array_equal`: Exact equality fails on tiny floating-point differences. Use a tolerance.

B19-L5 exercise 4

Hints. (1) `counts = rng.binomial(n, p, size=sims)` gives one success count per simulation. (2) For each count, `lo, hi = ci_fn(int(count), n)`, then check the **true** p: `lo <= p <= hi`. (3) Return the number covered divided by sims. Check sims first.

Answer.

```python
from collections.abc import Callable

import numpy as np


def coverage(
    ci_fn: Callable[[int, int], tuple[float, float]],
    rng: np.random.Generator,
    p: float,
    n: int,
    sims: int,
) -> float:
    """Share of simulated samples whose interval covers the true proportion.

    Args:
        ci_fn: Interval function taking (successes, n) and returning (lo, hi).
        rng: Seeded generator.
        p: True proportion.
        n: Sample size per simulation.
        sims: Number of simulations.

    Returns:
        The coverage, between 0 and 1.

    Raises:
        ValueError: If sims is below 1.
    """
    if sims < 1:
        raise ValueError("sims must be at least 1")
    counts = rng.binomial(n, p, size=sims)
    covered = 0
    for count in counts:
        lo, hi = ci_fn(int(count), n)
        if lo <= p <= hi:
            covered += 1
    return covered / sims
```

Coverage asks how often the interval contains the true value. An interval almost always contains its own estimate, so checking the estimate instead of p hides a broken method.

B19-L5 exercise 5

Hints. (1) Where does each expected value come from? (2) Remove chance rather than retrying around it. (3) Size the tolerance from theory.

Answer.

- An independent oracle for the exact value.
- A seeded generator, never retries.
- A coverage band sized from the standard error.
- Every edge case the function documents.

An oracle, a seed, a theory-based tolerance and the documented edge cases make a statistical test both stable and sharp.

Common wrong answers.

- exact = `copy`: A value copied from the function checks only that it agrees with itself.
- random = `retry`: Retries hide failures. Remove chance with a seed instead.
- coverage = `pvalue`: A p-value threshold fails by chance one run in twenty, or passes a broken method when the simulation is small.

B19-L6 Testing machine learning pipelines

B19-L6 exercise 1

Hints. (1) Which assertion changes when the scaler sees test rows? (2) What does a fitted StandardScaler store? (3) Its means.

Answer. a) Fit on the training split, then assert the scaler's `mean_` equals `X_train.mean(axis=0)`. Because: r1) It checks what the pipeline learned, which differs exactly when test rows leak in.

A scaler fitted on all rows stores the overall mean. Comparing its `mean_` with the training mean detects that directly.

Common wrong answers.

- `b`: Leakage usually raises offline AUC, so this test passes more easily, not less.
- `c`: Probabilities sum to 1 with or without leakage.
- `d`: A leaky pipeline fits without errors.
- `a|r2`: Right test, wrong reason. It works because it inspects the fitted state, not because of speed.

B19-L6 exercise 2

Hints. (1) Which rows does fit see? (2) Does predict change what the scaler learned? (3) (1 + 3 + 5 + 7) / 4.

Answer.

```
4.0
```

`fit` learns the mean from the four training rows, 4.0. `predict` only applies it, so the stored mean stays 4.0.

Common wrong answers.

- `23.2`: That is the mean of all five values. The scaler learned from the four training rows only. predict uses it and changes nothing.
- `100.0`: predict transforms the test row with the training mean. It does not refit the scaler.

B19-L6 exercise 3

Hints. (1) A DummyClassifier strategy that predicts the class shares. (2) AUC needs probabilities. (3) `prior` and `predict_proba`.

Answer.

```
baseline = DummyClassifier(strategy="prior").fit(X_train, y_train)
base_auc = roc_auc_score(y_test, baseline.predict_proba(X_test)[:, 1])
model_auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
assert model_auc > base_auc + 0.2
```

`DummyClassifier(strategy="prior")` gives every row the same scores, so its AUC is 0.5. The model must beat that by a clear margin.

Common wrong answers.

- g1 = `prior`; g2 = `predict`: AUC needs scores, not class labels. Use predict_proba and take column 1.

B19-L6 exercise 4

Hints. (1) `perm = rng.permutation(len(X))`, then `X[perm]` is a shuffled copy. (2) Row i of `model.predict_proba(X[perm])` belongs to original row `perm[i]`. (3) Compare with `original[perm]` using `np.allclose`, and return a bool.

Answer.

```
import numpy as np


def order_invariant(model: object, X: np.ndarray, rng:
    np.random.Generator) -> bool:
    """True when each row's predicted probabilities do not depend on row
    order.
```

```
6
7     Args:
8         model: Fitted estimator with predict_proba.
9         X: Feature matrix.
10        rng: Seeded generator for the permutation.
11
12    Returns:
13        Whether shuffling the rows leaves each row's probabilities
      unchanged.
14    """
15    perm = rng.permutation(len(X))
16    original = model.predict_proba(X)
17    shuffled = model.predict_proba(X[perm])
18    return bool(np.allclose(shuffled, original[perm]))
```

Indexing the original predictions with the same permutation lines every row up with its shuffled twin. Sorting the scores instead would hide a model that swaps them between rows.

B19-L6 exercise 5

Hints. (1) What differs between two runs of this test? (2) Check every call that draws random numbers for a seed. (3) One of them has none.

Answer. The bug is on line 9.

Fixed version:

```
1  from sklearn.datasets import make_classification
2  from sklearn.ensemble import RandomForestClassifier
3  from sklearn.metrics import roc_auc_score
4  from sklearn.model_selection import train_test_split
5
6
7  def test_model_beats_threshold():
8      X, y = make_classification(n_samples=300, random_state=0)
9      X_train, X_test, y_train, y_test = train_test_split(X, y,
       test_size=0.3, random_state=0, stratify=y)
10     model = RandomForestClassifier(n_estimators=50,
       random_state=0).fit(X_train, y_train)
11     auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
12     assert auc > 0.97
```

The split draws a new random set of test rows each run, so the AUC varies. Seed the split, and stratify it so both classes appear in the same shares.

Common wrong answers.

- Line 10: The forest has random_state=0, so it is deterministic for a given split.
- Line 8: make_classification has random_state=0, so the data is the same every run.

- Line 12: A fixed bar is fragile, but it would fail every run or none. Something changes between runs.

B19-L7 Isolate I/O with fakes, monkeypatch and mocks

B19-L7 exercise 1

Hints. (1) `from requests import get` copies the name into fetch.py. (2) Which name does load() look up when it runs? (3) The one in fetch.

Answer. b) `patch("fetch.get")` Because: r1) fetch.py looks up the name get in its own module, so that is the name to replace.

`from requests import get` creates the name `fetch.get`. Replace that name, where the code looks it up.

Common wrong answers.

- `a`: fetch.get still points at the original function, so the test makes a real request.
- `c`: patch needs a full dotted path to an importable name.
- `d`: fetch.py never imports the requests module itself, so there is no fetch.requests to patch.
- `b|r2`: Right patch, wrong reason. It works because that is where the code looks the name up.

B19-L7 exercise 2

Hints. (1) side_effect as a list: one item per call, in order. (2) An exception in the list is raised instead of returned. (3) Does a call that raises still count?

Answer.

```
10
error: down
2
```

The first call returns the dict, the second raises the ConnectionError, and the mock records both calls.

Common wrong answers.

- `10 error: down 1`: call_count counts every call, including the one that raised.
- `{'n': 10} error: down 2`: The first call returns the dict, and `["n"]` takes its value, 10.

B19-L7 exercise 3

Hints. (1) One test removes the variable, the other sets it. (2) Both methods end in `env`. (3) `delenv` and `setenv`.

Answer.

```python
def test_missing_token_raises(monkeypatch):
    monkeypatch.delenv("SURVEY_API_TOKEN", raising=False)
    with pytest.raises(RuntimeError, match="SURVEY_API_TOKEN"):
        load_token()


def test_token_is_read(monkeypatch):
    monkeypatch.setenv("SURVEY_API_TOKEN", "test-token")
    assert load_token() == "test-token"
```

`monkeypatch.delenv` and `monkeypatch.setenv` change the environment for one test, and pytest restores it afterwards.

Common wrong answers.

- g1 = `setenv`; g2 = `setenv`: The first test needs the variable gone. `delenv` removes it, and `raising=False` allows it to be missing already.

B19-L7 exercise 4

Hints. (1) Add `get` as a parameter and call it where `requests.get` was. (2) Check `response.status_code != 200` before reading the body. (3) Wrap the count in `int(...)`, and import `Callable` from `collections.abc` and `Any` from `typing`.

Answer.

```python
from collections.abc import Callable
from typing import Any

API = "https://survey.example/api/waves"


def wave_count(wave_id: str, get: Callable[..., Any]) -> int:
    """Number of respondents in one wave, from the survey API.

    Args:
        wave_id: Wave to look up.
        get: HTTP GET function, such as requests.get.

    Returns:
        The respondent count.

    Raises:
        RuntimeError: If the API does not answer with status 200.
    """
```

```
    response = get(f"{API}/{wave_id}", timeout=30)
    if response.status_code != 200:
        raise RuntimeError(f"wave API returned {response.status_code}")
    return int(response.json()["respondents"])
```

With `get` as a parameter, a test passes a fake response for each case: a normal count and an error status. Production passes `requests.get`.

B19-L7 exercise 5

Hints. (1) Read the first line of reports.py. (2) Which name does wave_count look up when it runs? (3) `reports.get`, not `requests.get`.

Answer. The bug is on line 14.

Fixed version:

```
# reports.py contains:
#     from requests import get
#
#     def wave_count(wave_id):
#         url = f"https://survey.example/api/waves/{wave_id}"
#         return get(url, timeout=30).json()["respondents"]

from unittest.mock import patch

import reports


def test_wave_count_reads_respondents():
    with patch("reports.get") as fake_get:
        fake_get.return_value.json.return_value = {"respondents": 812}
        assert reports.wave_count("W14") == 812
```

`from requests import get` binds `reports.get` to the real function at import. Patching `requests.get` changes a different name. Patch `reports.get`, or pass `get` in as a parameter.

Common wrong answers.

- Line 15: Setting the fake's return value is fine. The fake is never used, because of which name it replaced.
- Line 10: Importing reports is needed. The patch target is the problem.

B19-L8 Coverage, markers and a test suite you can trust in CI

B19-L8 exercise 1

Hints. (1) Every line ran. What did not happen? (2) An if has two ways out. (3) Which report counts both?

Answer. a) `--cov-branch` Because: r1) Branch coverage reports each if that never went one of its two ways, even when every line ran.

`--cov-branch` reports a missed jump such as `2->4` for an if whose false path never ran, even at 100% line coverage.

Common wrong answers.

- `b`: Line coverage was already 100%, so the floor passes. Lines ran. A path did not.
- `c`: That deselects slow tests. It measures nothing.
- `d`: Strict markers catch marker typos, not untested paths.
- `a|r2`: Right setting, wrong reason. Branch coverage adds measurement. It does not speed anything up.

B19-L8 exercise 2

Hints. (1) Which lines run when cap is 3? (2) The if has two branches. How many were taken? (3) Branch coverage counts statements and branches together: (4 + 1) / (4 + 2).

Answer.

```
100 83
```

All four statements run, so line coverage is 100%. With branches, 5 of 6 items are covered, 83%, and the report lists the missed jump `2->4`.

Common wrong answers.

- `75 67`: Every line runs when cap=3: the if, the assignment and the return. Line coverage is 100%.
- `100 100`: The if never takes its false path, the jump from line 2 straight to line 4. Branch coverage counts that.
- `100 50`: pytest-cov counts statements and branches together: 4 of 4 statements plus 1 of 2 branches is 5 of 6, 83%.

B19-L8 exercise 3

Hints. (1) The flag names what it makes strict. (2) The marker name goes before the colon. (3) `--strict-markers` and `slow`.

Answer.

```
[tool.pytest.ini_options]
addopts = "--strict-markers"
markers = ["slow: takes more than a few seconds"]
```

Registered markers document themselves, and `--strict-markers` turns a typo into an error instead of a silently unused marker.

Common wrong answers.

- g1 = `--strict`; g2 = `slow`: `--strict` is not the option name here. The command-line flag is `--strict-markers`. In config, pytest 9 also offers `strict = true`.
- g1 = `-m slow`; g2 = `slow`: `-m slow` would run only slow tests on every run. Registering markers strictly is `--strict-markers`.

B19-L8 exercise 4

Hints. (1) Four return statements need at least four cases, including None. (2) A score of 0.9 runs the high branch but cannot tell `>=` from `>`. (3) Test exactly 0.8 and exactly 0.5, the values where the two versions disagree.

Answer.

```python
import pytest


@pytest.mark.parametrize(
    ("score", "expected"),
    [
        pytest.param(None, "unknown", id="missing"),
        pytest.param(0.8, "high", id="at-0.8"),
        pytest.param(0.79, "medium", id="below-0.8"),
        pytest.param(0.5, "medium", id="at-0.5"),
        pytest.param(0.49, "low", id="below-0.5"),
    ],
)
def test_risk_band(score: float | None, expected: str) -> None:
    assert risk_band(score) == expected
```

Cases at 0.9, 0.6, 0.1 and None reach every line, yet miss both off-by-one changes. Only values on the thresholds tell `>=` from `>`. Coverage shows what ran. Mutants show what was checked.

B19-L8 exercise 5

Hints. (1) Where must everything run? (2) What keeps the local loop fast? (3) Fix causes, raise floors gradually.

Answer.

- Pull requests run everything, with branch coverage and a floor.
- Locally, deselect slow tests and iterate with -x and --lf.
- Flaky tests get a root cause, never a retry.
- The floor starts where the team is and only rises.

CI runs the whole suite with branch coverage and a rising floor. Developers run the fast subset. Flaky tests are fixed at the cause.

Common wrong answers.

- flaky = `retry`: A retry hides the failure and whatever causes it. Seeds, isolation and fakes remove the cause.
- floor = `hundred`: A 100% demand produces tests that run lines and assert nothing. Set a floor and raise it.
- local = `full`: A slow loop makes people stop running tests. Keep the full run for CI.